Sardar Patel Institute of Technology
Bhavan's Campus, Munshi Nagar, Andheri (W), Mumbai : 400058, India
(An Empowered Autonomous College Affiliated to University of Mumbai)
Department of Computer Engineering

Name: Jaynish Thakar **UID:** 2024300260

Name: Lavya Surana **UID:** 2024300251

Division: D **Batch:** B

MAMRS Project Implementation Report

Mood and Action Based Music Recommendation System

Project Overview and Objectives

MAMRS is a web application that recommends songs using a listener's selected mood, planned activity and saved music preferences. It combines contextual information with preferred artists, genres and languages, then presents ranked tracks with explanations. The implemented prototype also stores ratings and helpfulness feedback so that subsequent recommendations can reflect the listener's responses.

The main objective is to make music discovery more relevant to a particular user and situation. For example, a request for Romantic music while Driving should respect the selected context and required language, while giving priority to songs that match the listener's favorite artists and genres. The two central implemented modules are Mood and Activity Based Song Recommendation, and Ratings and Feedback.

Problem Addressed

The initial catalog was small, and imported metadata could contain incomplete labels, repeated titles and mixed artist credits. These issues affect both candidate selection and the credibility of the results. A track imported from an artist's release is not automatically a suitable mood or activity match. Collaborations and compilation releases also require careful handling so that unrelated artists are not presented as favorites.

The implementation addresses these problems through preference aware ranking and a separate catalog ingestion workflow. Imported recordings are validated, normalized and staged before they become eligible for contextual recommendations. Missing labels remain visible as a quality issue rather than being silently replaced with guessed values.

Application Architecture

The browser interface sends requests to a Node.js backend. The backend manages accounts, preferences, catalog access, recommendation generation and feedback persistence. SQLite supports local operation, while a PostgreSQL adapter provides a path to a Supabase database connection. The ingestion worker runs as a separate background process against the configured database, allowing catalog updates to continue independently of interactive recommendation requests.

Core Modules and User Workflow

Mood and Activity Based Song Recommendation

A user signs in, saves music preferences, selects a mood and activity, and requests a mix. The server loads the catalog visible to that user together with saved preferences and rating history. The recommendation engine is a deterministic content based system: it evaluates track metadata and contextual labels through explicit rules, rather than training a machine learning model.

Mood and activity constraints are kept strict. Language selections are required by default, with an option to allow broader discovery. Artist and genre preferences influence the ordering of eligible candidates, including matches found in structured collaboration credits and normalized genre aliases. Tracks with more preference matches take priority; ranking points order tracks with the same match count.

Scoring combines contextual relevance, artist and genre matches, language, personal rating history and helpfulness feedback. The engine also prefers a varied set of artists, using a soft preference for no more than two tracks from the same artist. Results display reasons so that a user can understand why a track was suggested. Ranking points represent relative relevance, not a probability or an experimentally measured accuracy.

When a favorite imported recording lacks context labels, the interface identifies it as awaiting context tags. The user can add personal tags or explore it without a selected context. Such a track is not represented as a verified match for the current mood and activity, and it is not included in a saved contextual mix. This distinction prevents missing metadata from producing misleading recommendations.

Ratings and Feedback

Users can give a recommended song a rating from 1 to 5, enter an optional comment, and mark the recommendation as helpful or unhelpful. The server validates the submission and saves it per user and song. A user can rate songs that their account has previously received as recommendations, which connects feedback to an actual interaction.

On the next recommendation request, the engine reads the updated feedback. A direct rating on a song takes priority over indirect signals from related artists or genres. Positive and negative responses therefore influence future scores for that user. The weighting rules remain fixed; the system does not learn new model parameters, and stored comments are not analyzed for sentiment.

Supporting Features and Interaction Sequence

Authentication, a personal catalog, preference editing, playlists, saved mixes and recommendation history support the two core modules. A typical sequence is: sign in; save artists, genres and languages; choose a context; generate recommendations; inspect track explanations; save a mix or playlist; submit feedback; and refresh recommendations. Saved snapshots retain the results of a request, while personal annotations remain associated with the user.

Catalog Ingestion and Data Quality

Discovery and Recording Identity

The ingestion pipeline accepts configured artists, recording files and application imports. MusicBrainz and Discogs adapters retrieve provider metadata, preserving source identifiers and structured artist credits. The India first configuration prioritizes Hindi, Punjabi and Tamil artists, with global English coverage as an additional goal. Artist discovery is an identity lookup, not a popularity chart; ambiguous names require a specific provider artist identifier.

Discogs discovery resolves releases and imports individual tracklist entries. Compilation tracks are included only when the selected artist appears in the track credits. MusicBrainz recording lookups retain recording identities, credits and available ISRCs. Provider records are retrieved again during import instead of accepting browser supplied metadata as authoritative.

Validation and Duplicate Handling

Imported recordings are checked for valid titles, artists and identifiers. Source identifiers take priority when recognizing a repeated import. Cross source duplicate detection requires matching normalized titles, artist names and durations; live, remix and version distinctions are preserved. Fuzzy or title only merging is not performed automatically, reducing the risk of combining different recordings that happen to share a name.

The catalog separates ready recordings from records that are pending context labels or need review. Malformed inputs are quarantined with their original data and an error reason, while valid records in the same batch can continue through staging. Existing song identifiers preserve references from ratings, playlists and personal annotations during cleanup and publication.

Label Enrichment and Human Review

Enrichment records label evidence and its source. Exact MusicBrainz mood and activity taxonomy tags can supply context labels when supported by at least three positive votes. These scores describe community vote strength and remain subjective. A recognized lyrics language can be obtained when exactly one linked performed work provides one recognized language. Multiple languages, medleys and unresolved cases remain unknown.

Genre, artist country and song title are not used to guess mood or language. Discogs provides credits and genre or style metadata but does not provide the same context enrichment. Recordings without suitable evidence remain pending. An operator can export a review batch, confirm language, supply mood and activity labels, record the label source, and publish only entries explicitly marked as reviewed. Reviewed labels take precedence during later provider refreshes.

Dynamic Processing and Database Workflow

Persistent jobs separate discovery, provider pages and recording enrichment. A background worker processes bounded batches or runs continuously. Worker leases prevent concurrent claims, expired jobs recover, and transient failures retry with backoff while respecting provider retry instructions. Refresh passes reuse existing identities and preserve reviewed labels. Configuring the server and worker with a PostgreSQL connection enables the same workflow for Supabase, but production scheduling and live deployment validation remain separate tasks.

Validation Results and Project Demonstration

Evidence from the Recorded Walkthrough

The demonstration uses an isolated account and starter catalog to show the implemented interaction. The user saves Bollywood, Arijit Singh and Hindi preferences, enables required language, and requests Romantic and Driving recommendations. The demonstration shows eligible tracks, ranking points and explanations, followed by feedback submission and a refreshed result.

With the same context and preferences, Tum Hi Ho increases from 86 to 92 ranking points after a 5 out of 5 rating and helpful feedback. This confirms that persisted personal feedback affects the scoring path in the demonstrated case. It does not establish recommendation accuracy across a large audience. The recorded validation reports 40 passing automated tests, and no browser JavaScript errors were observed during the captured walkthrough.

Current Implementation Boundaries

The prototype recommends metadata records and does not provide audio playback or downloads. It does not automatically detect a user's mood, train a collaborative filtering model or perform sentiment analysis on comments. The video describes the ingestion workflow, but it does not demonstrate a live provider import or a production Supabase upload. Comprehensive Spotify coverage and popularity rankings have not been implemented.

The current server ranks the catalog visible to the user in application memory. A substantially larger catalog will require indexed database candidate retrieval and paginated preference options. Production PostgreSQL behavior, worker hosting and provider quota management also require deployment validation. The successful local demonstration should therefore be read as evidence of the implemented workflow, rather than a production scale performance assessment.

Further Development

The next stage is to expand reviewed Hindi, Punjabi, Tamil and global English recordings, while tracking label coverage and duplicate rates. Spotify availability resolution and a permitted popularity data source would help prioritize recognizable songs. Candidate retrieval should be moved toward indexed database queries as the catalog grows. Evaluation with users should measure preference match rates, relevance ratings and the effect of feedback before broader deployment.

Project Demonstration Video

The four minute video presents the application interface, the two core modules and the feedback driven ranking example. It is stored in the Google Drive folder provided with this submission.

Open the project demonstration video folder on Google Drive

Live application: mamrs.jaynishthakar.tech

GitHub repository: https://github.com/jaynishthakar/mamrs.git